

EEBus SPINE Implementation Guideline

Protocol and Resource Guidelines

Version 1.0.0

Cologne, 2026-07-16

EEBus Initiative e.V.

Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

Terms of use for publications of EEBus Initiative e.V.**General information**

The specifications, particulars, documents, publications and other information provided by the EEBus Initiative e.V. are solely for general informational purposes. Particularly specifications that have not been submitted to national or international standardisation organisations by EEBus Initiative e.V. (such as DKE/DIN-VDE or IEC/CENELEC/ETSI) are versions that have not yet undergone complete testing and can therefore only be considered as preliminary information. Even versions that have already been published via standardisation organisations can contain errors and will undergo further improvements and updates in future.

Liability

EEBus Initiative e.V. does not assume liability or provide a guarantee for the accuracy, completeness or up-to-date status of any specifications, data, documents, publications or other information provided and particularly for the functionality of any developments based on the above.

Copyright, rights of use and exploitation

The specifications provided are protected by copyright. Parts of the specifications have been submitted to national or international standardisation organisations by EEBus Initiative e.V., such as DKE/DIN-VDE or IEC/CENELEC/ETSI, etc. Furthermore, all rights to use and/or exploit the specifications belong to the EEBus Initiative e.V., Deutz-Mülheimer-Straße 183, 51063 Cologne, Germany and can be used in accordance with the following regulations:

The use of the specifications for informational purposes is allowed. It is therefore permitted to use information evident from the contents of the specifications. In particular, the user is permitted to offer products, developments and/or services based on the specifications.

Any respective use relating to standardisation measures by the user or third parties is prohibited. In fact, the specifications may only be used by EEBus Initiative e.V. for standardisation purposes. The same applies to their use within the framework of alliances and/or cooperations that pursue the aim of determining uniform standards.

Any use not in accordance with the purpose intended by EEBus Initiative e.V. is also prohibited.

Furthermore, it is prohibited to edit, change or falsify the content of the specifications. The dissemination of the specifications in a changed, revised or falsified form is also prohibited. The same applies to the publication of extracts if they distort the literal meaning of the specifications as a whole.

It is prohibited to pass on the specifications to third parties without reference to these rights of use and exploitation.

It is also prohibited to pass on the specifications to third parties without informing them of the authorship or source.

Without the prior consent of EEBus Initiative e.V., all forms of use and exploitation not explicitly stated above are prohibited.

Table of contents

Table of contents.....	3
List of tables	3
1 Scope of the document	4
1.1 References	4
1.1.1 EEBUS documents.....	4
1.1.2 Normative references.....	4
1.2 Terms and definitions	4
1.3 Requirements.....	5
1.3.1 Requirements wording	5
1.3.2 Notes on requirements in this document	5
2 Clarifications on the SPINE Protocol Specification	6
2.1 Completeness of protocol data.....	6
2.2 Usage of the "useCaseAvailable" flag in Use Case discovery.....	6
2.3 Origin and destination of binding and subscription calls.....	7
2.4 Minimization of unneeded notifications	7
2.4.1 Definition of "unneeded notifications"	7
2.4.2 Rationale and best practice	8
2.5 Strict adherence to version formatting	8
2.6 Best practices for error handling and unresponsive peers.....	9
2.6.1 Differentiating expected errors (NACK) from unresponsiveness	9
2.6.2 Escalation path for unanswered messages (retry mechanism).....	9
2.6.3 Handling discovery phase failures	10
2.6.4 Partial degradation and user notification.....	10
2.6.5 Usage of "errorNumber"	10
2.7 Clarification on communication modes and mandatory device addresses.....	11
3 Clarifications on the SPINE Resource Specification.....	12
3.1 Completeness of resource data	12
3.2 Usage of "scaledNumber"	12
3.2.1 Modified requirement for ScaledNumberType	12
3.3 Partial updates (via RFE)	12
3.4 Uniqueness of Feature Types per Entity.....	14
3.5 Requirements for the usage of error numbers in a result message	15

List of tables

Table 1: Interpretation of "number" and "scale" in non-RFE and RFE messages	13
Table 2: Values specified for element errorNumber.....	15

1 Scope of the document

This document is an implementation guideline for the Smart Premises Interoperable Neutral-message Exchange (SPINE) (short-name for this document: IG-SPINE).

It details how to interpret specific concepts, structural elements and communication sequences of the SPINE protocol and gives examples and best practices on topics reported as probably difficult to understand, incomplete or where misinterpretations could occur.

Furthermore, this guideline contains specific modifications that intentionally deviate from the current specifications for practical or technical reasons. These retroactive changes to the existing specification are additionally covered by separate errata sheets, whereas the general clarifications are exclusively documented in this implementation guide.

The sections of this document clarify the SPINE Technical Specifications. They are, together with the core specifications, the basis for the implementation and testing procedures of EEBUS devices.

This document will be continuously updated as new issues requiring clarification or correction are identified.

Chapter 2 focuses on clarifications on the SPINE Protocol Specification.

Chapter 3 focuses on clarifications on the SPINE Resource Specification.

1.1 References

1.1.1 EEBUS documents

[SPINE] *Minimum:* EEBus_SPINE_V1.3.0_Final.zip

[SHIP] *Minimum:* EEBus_SHIP_TS_Specification_v1.0.1.pdf
Recommended: EEBus_SHIP_TS_Specification_v1.1.0.pdf

[UC-IG-General] EEBus_UC_IG_GeneralGuidelines_V1.0.0.pdf

1.1.2 Normative references

[RFC2119] IETF RFC 2119: 1997, Key words for use in RFCs to indicate requirement levels. Please see section 1.3.1 for details.

1.2 Terms and definitions

ACK

Abbreviation for "Acknowledgement"

Actor

An Actor models a role within a Use Case definition (e.g. an energy manager or a heat pump).

72 EMS

73 Abbreviation for "Energy Management System"

74 IG-SPINE

75 Abbreviation for "Implementation Guideline for SPINE" (short name of this document)

76 NACK

77 Abbreviation for "No Acknowledgement"

78 Scenario

79 Part of a Use Case. Splitting a Use Case into Scenarios helps readers understand the Use Case more
80 quickly. Some Scenarios are mandatory for a Use Case, whereas others may be recommended or
81 optional.

82 SPINE

83 Abbreviation for "Smart Premises Interoperable Neutral-message Exchange": Technical Specification
84 of EEBus Initiative e.V.

85 UC

86 Abbreviation for "Use Case"

87

88 1.3 Requirements**89 1.3.1 Requirements wording**

90 The following keywords are used:

- 91 - SHALL
- 92 - SHALL NOT
- 93 - SHOULD
- 94 - SHOULD NOT
- 95 - MAY

96 Note: They apply only if written in capital letters.

97 For the meaning of the keywords, please refer to [RFC2119].

98

99 1.3.2 Notes on requirements in this document

100 This document clarifies requirements of the [SPINE] Resource Specification or Protocol Specification.

101 Regarding these specifications, the keywords "SHALL" and "SHALL NOT" are generally already
102 covered by mandatory requirements of these specifications. However, in cases where this guideline
103 explicitly defines retroactive changes to the existing specification, these keywords express
104 mandatory requirements that intentionally override the original specifications. Such modifications
105 are additionally tracked via errata sheets.

106

2 Clarifications on the SPINE Protocol Specification

2.1 Completeness of protocol data

The SPINE Protocol Specification defines the fundamental structure and message formats of the SPINE protocol. It is strictly required that all mandatory Elements defined in the specification are present in the communication. Incomplete SPINE protocol data is not permitted.

While a comprehensive list of all mandatory elements is out of scope for this document, a common example can be found in the basic message structure. For instance, mandatory elements within the header of a SPINE message (such as the "msgCounter") SHALL always be provided.

A recipient of an incomplete SPINE message MAY discard it^{*1}. However, it should be noted that this may prevent communication with older devices in the field.

In case the SPINE header misses mandatory Elements or has incorrect values, the message SHOULD be discarded silently in any case (i.e., without any response, regardless of the kind of message).

^{*1}: In the case of messages that require a response from the recipient ("read", "write" or possibly "call"), an appropriate error message SHALL be returned, but only if the SPINE header was correctly set (see above).

2.2 Usage of the "useCaseAvailable" flag in Use Case discovery

The Use Case discovery process utilizes the "useCaseAvailable" flag to indicate whether a specific Use Case is currently supported by an actor. According to the SPINE Protocol Specification, if this flag is omitted, a default value of "true" is assumed.

While the original specification introduces this flag in the context of Client Actors^{*1}, it does not explicitly forbid Server Actors from transmitting it. This ambiguity has led to mixed implementations in the field where Server Actors^{*1} also set this flag. To clarify the intended architectural behavior and prevent interoperability issues, the following rules apply:

Client Actors: The "useCaseAvailable" flag is intended for Client Actors to express their current Use Case availability. Only Client Actors may set this flag to "false".

Server Actors: Server Actors SHOULD NOT set the "useCaseAvailable" flag at all. If a Server Actor does set the flag (e.g., due to legacy implementation reasons), it SHOULD only set it to true. A Server Actor SHALL NOT set the flag to false.

Handling of incoming flags: To ensure robust behavior against legacy Server Actor implementations that might still transmit this flag, Client Actors SHALL completely ignore any "useCaseAvailable" flag sent by a Server Actor.

^{*1}: In the existing SPINE and Use Case documentation, there is no definition of "Client Actors" or "Server Actors". This definition is added in the [UC-IG-General].

2.3 Origin and destination of binding and subscription calls

The SPINE Protocol Specification defines that all functions related to the binding and subscription concepts SHALL be implemented on the primary NodeManagement instance (Entity 0, Feature 0). However, varying implementations exist in the field regarding the source address used when transmitting such requests.

To ensure interoperability and clarify the expectations for both the sender and the receiver of binding and subscription messages, the following rules apply:

- header.addressDestination (sink): Any request regarding bindings or subscriptions (e.g., nodeManagementBindingRequestCall, nodeManagementSubscriptionRequestCall or read requests for binding/subscription data) SHALL be addressed to the recipient's primary NodeManagement instance. The addressDestination in the SPINE header SHALL specify Entity 0 and Feature 0.
- header.addressSource (origin): While the Protocol Specification permits these requests to originate from any source address, implementers SHOULD use their own primary NodeManagement instance (Entity 0, Feature 0) as the addressSource in the SPINE header. This represents the best practice, aligns with the intended architecture and significantly simplifies message routing and debugging.
- Tolerance of source addresses: Because the usage of the primary NodeManagement instance as the source address is explicitly permitted as an option ("MAY") in the Protocol Specification, a receiving device SHALL NOT reject or ignore a valid binding or subscription request solely because it originates from a feature address other than Entity 0, Feature 0. Receivers SHALL process these requests correctly regardless of the used addressSource.
- Response addressing (symmetry): When generating a response to a request (e.g., a reply or resultData message), the responding device SHALL use the addressSource of the incoming request as the addressDestination for its response. A device SHALL NOT hard-code the response destination to Entity 0, Feature 0 if the original request originated from a different feature address.

2.4 Minimization of unneeded notifications

The SPINE protocol utilizes a subscription mechanism where server actors send notify messages to inform subscribed clients about data updates. While the protocol supports real-time data exchange, it is crucial to maintain network efficiency and system stability.

Requirement: Implementations SHOULD minimize unneeded notifications.

2.4.1 Definition of "unneeded notifications"

A notification is generally considered unneeded or excessive if it falls into one of the following categories:

1. Unchanged data: Transmitting a notify message when the underlying data has not changed (e.g., sending the exact same temperature measurement repeatedly every 100 milliseconds).

2. Excessive frequency (micro-fluctuations): Transmitting notifications at extremely high frequencies (e.g., every millisecond) simply because a highly volatile value (such as active power) experiences negligible micro-fluctuations.

2.4.2 Rationale and best practice

Minimizing unnecessary traffic is essential to prevent network congestion and to reduce the processing burden on receiving nodes. An energy management system (EMS), for example, typically communicates with multiple devices simultaneously. If several devices flood the network with sub-second notifications, it can easily overload the processing capabilities of the EMS. This can lead to dropped messages, delayed control reactions and overall system instability.

Best practice: Devices SHOULD implement sensible rate limits (e.g., a maximum update frequency) and/or deadbands (a minimum value delta required before a change is considered significant enough to trigger a notification) for highly volatile data like measurements.

2.5 Strict adherence to version formatting

Within SPINE, different version information strings are exchanged to ensure protocol and data model compatibility between actors. This includes versioning of SPINE itself, as well as on Use Case level.

A common cause for interoperability issues is the transmission of version strings that do not comply with the defined formatting rules (e.g., because of internal firmware build strings, unescaped characters or custom versioning schemes).

Requirement: All transmitted version strings SHALL match exactly the formatting rules and regular expressions defined in the underlying specifications. Applications in the field that receive version strings that do not comply to the formatting rules MAY abort the communication with that other device.

To ensure compliance, implementers must strictly verify the used version strings against the format definitions located in the following specification sections:

- Use Case discovery:
 - During the Use Case discovery phase, the "useCaseVersion" element within the nodeManagementUseCaseData SHALL exactly adhere to the defined schema type (SpecificationVersionType). Devices SHALL transmit the exact, official version number of the supported EEBUS Use Case.
 - In addition to the main version, the "useCaseDocumentSubRevision" field SHALL be set. It SHALL be formatted correctly according to the SPINE Protocol Specification.
- SPINE:
 - During the detailed discovery phase, a device SHALL present its supported versions of SPINE. Each stated version information SHALL exactly adhere to the version definition of the SpecificationVersionType.
 - In each message, the SPINE header includes the (selected) SPINE version. The format SHALL exactly adhere to the definition of the SpecificationVersionType.

2.6 Best practices for error handling and unresponsive peers

In real-world implementations, communication between devices may experience temporary or permanent disruptions. This section outlines best practices for handling scenarios where a connected peer does not respond as expected. The goal is to ensure a robust ecosystem by defining a clear escalation path, preventing premature connection drops and avoiding infinite retry loops that clog the network.

2.6.1 Differentiating expected errors (NACK) from unresponsiveness

Before triggering any escalation or retry mechanism, it is crucial to distinguish between an unresponsive peer (timeout) and a peer that actively rejects a command (NACK).

- Active rejection (NACK): If a device sends a write command and receives a resultData message containing an error code (e.g., because the device is currently in a mode that prevents the requested change), this is a valid and successful protocol exchange.
 - o Rule: A valid NACK SHALL NOT trigger a retry mechanism and SHALL NOT lead to a termination of the SHIP connection. The client must handle the error on the application level according to the Use Case logic.
- Unresponsiveness (timeout): If a device sends a message that requires a response (like a read or write command) and does not receive the expected reply or resultData within a reasonable application timeout^{*1}, the peer is considered unresponsive for this specific transaction.

Please see section 2.6.2 for a possible escalation path in case of an unresponsive communication partner.

^{*1}: The default timeout for a response is 10 seconds (see [SPINE] Protocol Specification, "maximum response delay").

2.6.2 Escalation path for unanswered messages (retry mechanism)

If a peer is considered unresponsive (timeout), the device SHOULD implement a staggered retry mechanism rather than immediately dropping the connection. Immediate disconnects lead to unstable network behavior.

The following escalation path is recommended as a best practice for handling unanswered messages during runtime:

1. First retry: If the initial message times out, retry the transmission after a short delay (e.g., 30 seconds).
2. Second retry: If the first retry fails, wait for a medium delay (e.g., 5 minutes) before the next attempt.
3. Third retry: If the second retry fails, wait for a longer delay (e.g., 15 minutes) before the final attempt.
4. Connection reset: If the message remains unanswered after the final retry, the device SHOULD close the underlying SHIP connection.

Closing the connection forces a clean slate. After closing the connection, the device SHOULD allow the remote peer to re-establish the SHIP connection, assuming the peer might have rebooted or recovered from its error state.

The "closing-side" can also try to re-establish the connection.

2.6.3 Handling discovery phase failures

A special edge case occurs if the unresponsiveness happens immediately after the SHIP connection is established, preventing the exchange of elementary discovery data (e.g., nodeManagementDetailedDiscoveryData or nodeManagementUseCaseData). Without this initial data, no Use Case can be executed.

- Escalation: The device SHOULD apply the staggered retry mechanism described in section 2.6.2 for the discovery requests.
- Blocking: If the SHIP connection is closed due to discovery timeouts and this cycle (connect -> discovery timeout -> retry path -> disconnect) repeats consecutively (e.g., three times), the device MAY block the peer from further automatic connection attempts to prevent resource exhaustion.

2.6.4 Partial degradation and user notification

Not all errors result in a complete communication breakdown. A device might successfully execute one Use Case (e.g., Limitation of Power Consumption) while another Use Case (e.g., Monitoring of Power Consumption) times out or fails persistently.

- Partial degradation: Devices SHOULD NOT close the entire SHIP connection if only a specific part of the data model or a single Use Case is unresponsive, as long as other critical Use Cases are still functioning correctly.
- User information: Automated processes (like closing connections or retries) handle the technical layer, but unresolved issues require human awareness. Devices SHOULD inform the user (e.g., via a smartphone app, web interface or local display) about degraded states.
 - o If a specific Use Case fails persistently, notify the user about the limited functionality.
 - o If a device is entirely blocked (as described in section 2.6.3), explicitly inform the user that the pairing process failed and prompt them to check the remote device or manually restart the pairing.

2.6.5 Usage of "errorNumber"

Please see section 3.5 for recommendations on the usage of the value within the Element "errorNumber".

2.7 Clarification on communication modes and mandatory device addresses

The SPINE Protocol Specification (chapter 6) defines two primary communication modes: the "simple communication mode" and the "enhanced communication mode".

Historically, the simple communication mode allowed the omission of the device component in the SPINE address structure, declaring it only as a "SHOULD" requirement. Conversely, the enhanced communication mode mandates the presence of the device component but also introduces complex routing mechanisms, such as destination lists and message forwarding to third-party devices.

In practical EEBUS implementations, the structural clarity of the enhanced mode is the de facto standard, while its complex routing features are rarely used in standard point-to-point SHIP connections.

To ensure strict interoperability and simplify message parsing across all protocol stacks, the following best practices and restrictions apply:

- Mandatory device component: The device component within the SPINE datagram header address structure (for both source and destination addresses) SHALL always be populated^{*1} (overriding the optionality permitted by the simple communication mode).
- Exclusion of complex routing: While the address structure must comply with the enhanced communication mode (by including the device component), implementers are NOT required to implement or support the advanced routing features, such as destination lists or the forwarding of SPINE messages to third devices, unless explicitly demanded by a specific system architecture or Use Case. Standard point-to-point communication over SHIP is sufficient.

^{*1}: The only exception is the initial read of the detailed discovery data where the device address of the peer is not known yet.

3 Clarifications on the SPINE Resource Specification

3.1 Completeness of resource data

The SPINE Resource Specification defines the data models, features and payloads used within SPINE messages. Similar to the protocol level, incomplete resource data is strictly prohibited. Every data element defined as mandatory in the respective resource data model*¹ SHALL be populated and transmitted.

This rule applies globally to all defined resources and data types. For instance, when a device transmits a `measurementListData` with one or more list entries, each of the `measurementData` structures SHALL at least contain the primary identifier `"measurementId"` and the `"value"`. Another example: Within a `loadControlLimitListData` entry, the `"value"` element has this rule: "If `isLimitActive` is set to `"true"`, the *value* SHALL be set. [...]".

*¹: Among others, such rules can be found in the "Feature Types" section of the [SPINE] Resource Specification.

3.2 Usage of "scaledNumber"

The `ScaledNumberType` is used throughout the SPINE Resource Specification to represent numeric values with decimal places, avoiding the issues associated with floating-point representations. A "scaledNumber" is calculated as $\text{value} = \text{number} * 10^{\text{scale}}$.

The original SPINE Resource Specification (section 3.10.1.8.1) defined that the `"number"` element SHALL be set and if the `"scale"` element is omitted, a default value of `"0"` is applied. However, due to severe misinterpretations this behavior is hereby modified.

3.2.1 Modified requirement for ScaledNumberType

To ensure robust communication and avoid misinterpretations caused by omitting elements, this Implementation Guideline enforces a strict deviation from the original default rule.

Rule: For any transmission of a `"scaledNumber"` (regardless of whether it is a reply, notify or write message and regardless of client or server role) both the `"number"` and the `"scale"` elements SHALL always be provided. The `"scale"` element is strictly mandatory.

Note: See section 3.3 for rules regarding `ScaledNumberType` in RFE messages.

Note: This overriding rule will be officially tracked via an Errata-Sheet.

3.3 Partial updates (via RFE)

The Restricted Function Exchange (RFE) is a mechanism in SPINE that allows devices to perform partial updates.

The interaction between RFE and the SPINE data model with default values (such as for `ScaledNumberType`) depends on the kind of the message exchange (RFE or full). To ensure

compatibility, a clear precedence is established between RFE merge logic and the SPINE Resource Specification rules.

The application of mandatory elements and default rules depends on whether a message represents a full data transfer or a partial data transfer via RFE.

1. Full data transfer: This applies to standard (non-RFE) read operations resulting in a full reply or a write command without RFE flags. In this case, all mandatory elements (e.g., "number" and "scale" within a value) SHALL be present. Omitted optional elements (e.g., "valueState") SHALL be interpreted using the default values defined in the SPINE Resource Specification.
2. Partial data transfer (RFE): This applies whenever the partial command option is used. In case RFE is used, the "merge logic" (omitted = retain existing state) takes precedence over any default rules. An omitted element SHALL be interpreted as "unchanged," and the previously stored value remains valid, even if the element has a defined default value.

The following table details how combinations of "number" and "scale" are interpreted in non-RFE and RFE messages:

Payload combination	Full data transfer (non-RFE)	Partial data transfer (RFE)
number & scale present	Standard calculation using both values.	Update both number and scale.
number present, scale missing	Invalid; scale is mandatory.	Update number, retain old scale.
number missing, scale present	Invalid; number is mandatory.	Retain old number, update scale.
number missing, scale missing	Invalid; number and scale are mandatory.	Retain both (no changes).

Table 1: Interpretation of "number" and "scale" in non-RFE and RFE messages

Clarification for "valueState" interpretation in RFE messages:

If a measurementListData entry with a "valueState" set to "error" was notified, the client SHALL store this information. If later, within a RFE message, only the "value" was notified, but not the "valueState", the client SHALL ignore the value's content as defined in the "value state rules". A notification with "valueState" set to "normal" is needed for the client to interpret the server's "value" (for this particular list entry) again. Note: If "valueState" is set to "outOfRange", the "value" SHALL be interpreted as being out of range and its content SHOULD be ignored as well.

Note on best practice for RFE payload structures:

While the RFE mechanism technically permits omitting unchanged sub-elements within a single list entry (e.g., within a specific xData element), devices SHOULD always transmit the completely populated list entry. RFE remains essential on the list level to update individual entries without transmitting the entire list (e.g., updating only one specific measurementData entry out of a large list). However, fully populating the transmitted entry with all relevant elements significantly reduces ambiguity and prevents state synchronization issues caused by complex deep-merge logic on the receiver's side.

3.4 Uniqueness of Feature Types per Entity

The SPINE architecture organizes functionality into Entities, which contain one or more Features. Each Feature is characterized by a specific "featureType" (e.g., Measurement, LoadControl, ElectricalConnection).

While the [SPINE] specification defines how features are grouped and addressed, it does not explicitly forbid the instantiation of multiple features with the identical "featureType" on the same Entity. In fact, descriptions regarding "Feature Groups" (which state that groups are not needed if features are only defined once on an entity) can lead to the misinterpretation that multiple features of the same type are permitted.

This Implementation Guideline strictly prohibits such implementations to ensure a deterministic mapping of functionality during the discovery process.

Rule: It is NOT permitted to have multiple Features with the same "featureType" within one and the same Entity.

Requirement: Each "featureType" SHALL occur at most once per Entity.

This is in line with the rule specified in each Use Case below the table containing the Actors's Features ("Table <X>: Feature Types and Functions used within this Use Case by the Actor <Y>"):

"For each of these Feature Types the following rule applies: There SHALL be at maximum one Feature with the Feature Type in the Entity."

Rationale and best practice:

If a client looks for a specific "featureType" (e.g., Measurement) on a server's Entity to execute a Use Case, the presence of two Measurement Features on that exact same Entity would create an unresolvable ambiguity.

If an implementation needs to provide multiple sets of data that belong to the same "featureType" (e.g., measuring power at different phases or points within the same physical entity), the implementation SHALL NOT create two separate Features of the same type.

Instead, one of the following approaches SHALL be used:

1. Multiple data sets: Use the single allowed Feature and provide multiple entries within its respective data lists (e.g., multiple measurementData entries with different "measurementIds" within one Measurement Feature).
2. Sub-Entities: If the functions are logically or physically distinct enough to warrant separate Features, they SHOULD be placed in separate sub-Entities.

3.5 Requirements for the usage of error numbers in a result message

The [SPINE] Resource Specification defines values for the Element "errorNumber" within the resultData Function, but without further details.

Some Use Cases define which errorNumber must be used in specific cases.

This section adds further requirements, for the usage of the "errorNumber".

Currently, the following "errorNumber" values are defined in [SPINE] (see Resource Specification, Table "Values specified for element errorNumber"):

Value	Description
0	No Error.
1	General Error.
2	Timeout.
3	Overload.
4	Destination unknown.
5	Destination unreachable.
6	Command not supported.
7	Command rejected.
8	Restricted function exchange combination not supported.
9	Binding is necessary for this command.

Table 2: Values specified for element errorNumber

Value 0: SHALL only be used if no error occurred during the receipt or evaluation of a message.

In all other cases, one of the following "errorNumber" values SHALL be used:

- Value 1: SHOULD be used, if an error occurred that fits no other category.
- Value 2: SHOULD be used, if a timeout occurred.
- Value 3: SHOULD be used, if the recipient encountered an overload that prevents the incoming message from being processed.
- Value 4: SHOULD be used, if the address stated in the SPINE header Element addressDestination is unknown to the recipient. Please also consider section 2.1.
- Value 5: SHOULD be used, if the address stated in the SPINE header Element addressDestination is not reachable for the recipient (this is only applicable if message forwarding within the enhanced communication mode is used).
- Value 6: SHOULD be used, if the command is not supported by the recipient (e.g., a read-request on a not supported Function).
- Value 7: SHOULD be used, if the command is rejected by the recipient (e.g., a write command on a non-writeable Function).
- Value 8: SHOULD be used, if the RFE combination of the received command is invalid.
- Value 9: SHOULD be used, if a client tries to write a Function without a proper binding on the according Feature of the recipient.